AURA — Patient Reactivation OS

Technical Handover Document — v1.1 MVP *Target audience: Claude Code (VS Code plugin)*

Table of contents

---

0. How to use this document

This is a build spec for an MVP, not a polished engineering reference. The product is a lean-startup play: ship the smallest correct thing, measure adoption, iterate. Every section is scoped to that goal.

If a requirement is not stated here, default to the simplest implementation that satisfies the v1 acceptance criteria in section 11. Do not pre-build for hypothetical scale. The architecture is intentionally chosen to allow additive evolution (email channel, real-time at scale, per-tenant DBs) without rewrites.

In-scope for this build:

- CSV upload that populates the `patients` table
- Inbox screen with two-way Twilio SMS (send + receive)

- Database schema future-proofed for: email channel, multi-tenant, super admin
- All other portal screens (dashboard, sequences, settings, patients list, etc.) generated as static UI per `index.html` reference — wiring is optional in this build pass

Out of scope for this build:

- Email sending (Resend or otherwise)
- Sequence execution engine (the 7-message 28-day flow)
- LLM-based reply intent detection (keyword matching is enough for v1)
- Booking integration adapters (Boulevard / Mangomint / Jane / Calendly)
- Reporting & digests
- A2P 10DLC registration — note in SOP, do not block dev

---

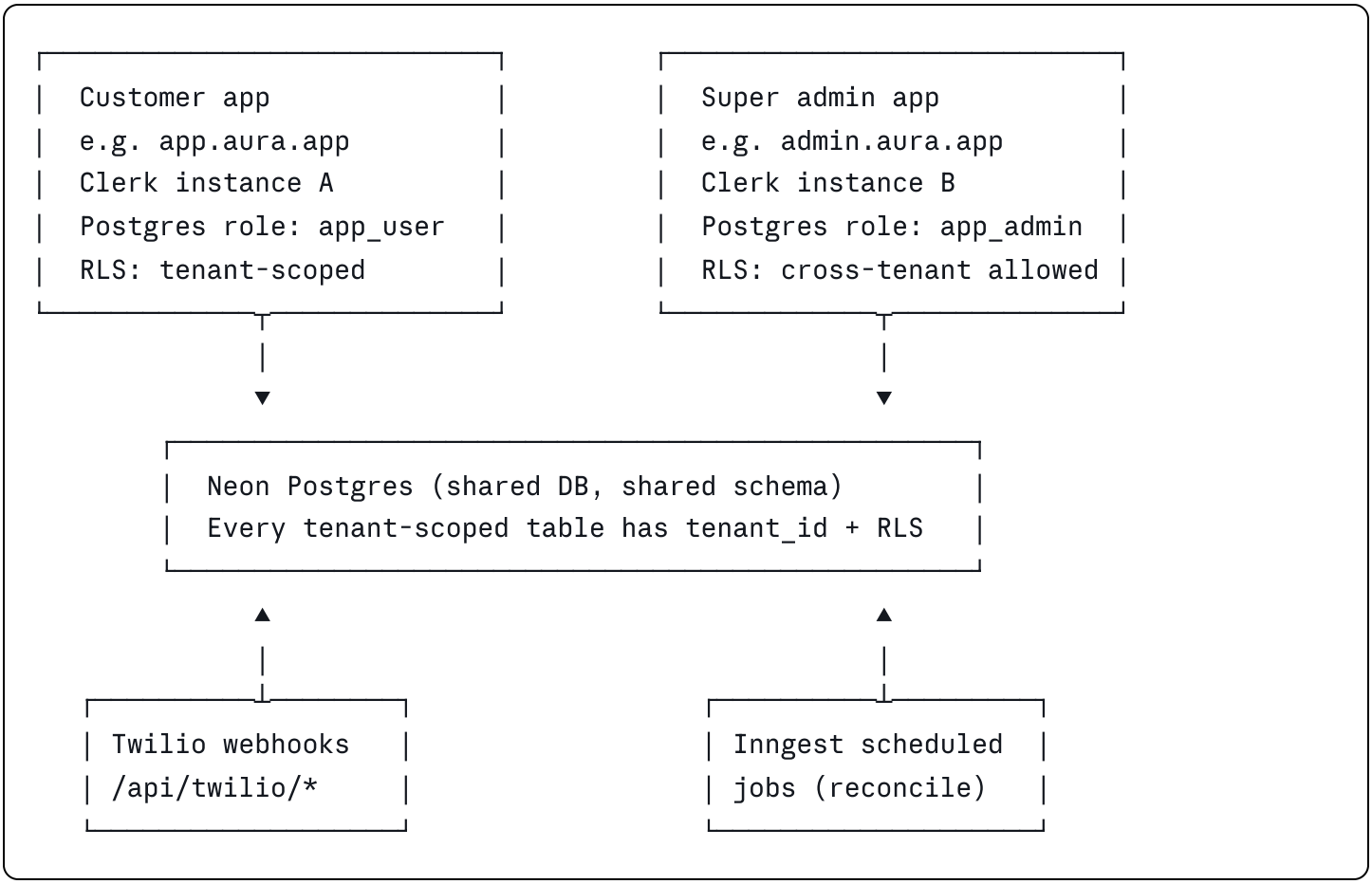
1. Architecture overview

1.1 Stack

Layer	Choice
Framework	Next.js 15 (App Router) + TypeScript, single full-stack app
UI	React Server Components + Client Components where interactive
Database	Neon Postgres (single instance, shared schema, RLS enforced)
ORM / migrations	Drizzle ORM (`drizzle-kit` for migrations)
Auth (customer app)	Clerk — Organizations = tenants (1:1)
Auth (admin app)	Clerk — separate Clerk instance, separate deployment
SMS	Twilio (programmable SMS, status callbacks, inbound webhook)
Background jobs	Inngest (used in v1 only for nightly Twilio status reconciliation)
Real-time inbox	Server-Sent Events (SSE) from a Route Handler
Hosting (initial)	Vercel — but kept Vercel-agnostic; see §1.4
Email channel	Deferred to v2 — use Resend when added

1.2 High-level shape

Two separate Next.js applications, two separate Clerk instances, one shared Neon Postgres database accessed under two different Postgres roles.



1.3 Why Next.js full-stack (and not a separate Node/Express BE)

One repo, one deploy, shared TypeScript types end-to-end. Server Components remove a whole class of API endpoints (the patients table renders server-side directly from Drizzle, no `/api/patients` needed). Auth is one Clerk middleware that wraps every request — UI pages, Server Actions, Route Handlers, Twilio webhooks. No CORS, no separate API client. SSE for the inbox is a Route Handler in the same app.

Business logic lives in pure service functions (Drizzle queries + plain TypeScript) under `lib/services/`. These do not depend on Next.js. If a decoupled Express service is needed later (e.g. to support a mobile app), those service functions move unchanged behind Express routes. The extraction is mechanical, not a rewrite.

1.4 Hosting portability (two protective rules)

Vercel is the initial host because it's the fastest path from `git push` to live URL. The architecture is **not** Vercel-locked. The following two rules keep the codebase portable to AWS (ECS, App Runner, Lambda via OpenNext), GCP (Cloud Run), Fly.io, Railway, Render, or any Docker host:

- **Rule 1:** No `@vercel/*` runtime imports in app code. Standard Next.js and Node APIs only. (Vercel Analytics via the optional script tag is fine.)
- **Rule 2:** `next.config.ts` sets `output: 'standalone'` from day one. A Dockerfile can build the app for any container host with no code changes.

Migration to AWS or GCP later: build the Docker image, push to the new host, repoint Twilio webhook URL, repoint Clerk allowed domain, set env vars. Neon, Clerk, Inngest, and Twilio do not move — they're SaaS that push to whatever URL you give them. Estimated migration effort: 1 day.

---

2. Multi-tenancy model

2.1 Decisions (locked-in)

- Shared database, shared schema. Every tenant-scoped table has a `tenant_id uuid not null` column.
- Postgres Row-Level Security (RLS) enforced at the database layer.
- Tenant context is set per-request via `SET LOCAL app.current_tenant = '<uuid>'` inside a transaction.
- Clerk Organizations = tenants. 1:1 mapping. Do not build a parallel tenant model in app code.
- Tenant resolution happens once per request in Next.js middleware, not in handlers.
- Schema-per-tenant or DB-per-tenant is deferred until a customer contractually requires it or noisy-neighbor issues appear.

2.2 Two Postgres roles

Role	Used by	Behavior
`app_user`	Customer-facing Next.js app	Subject to RLS tenant isolation policy. Every query sees only rows where `tenant_id = app.current_tenant`.
`app_admin`	Super admin app	Permissive RLS policy — can read across tenants. Writes are explicit, audited, and rare.

Each role is reached via a separate Neon connection string. The two apps cannot accidentally cross over.

2.3 Tenant resolution flow (customer app)

```
Request arrives
  └─ Clerk middleware runs
      └─ Resolves auth().userId and auth().orgId
          └─ orgId IS the tenant_id (1:1 mapping with Clerk Org)
              └─ All downstream code reads tenantId from auth context
                  └─ DB calls wrapped in withTenant(tenantId, async (tx) => { ...
})
                      which sets SET LOCAL app.current_tenant inside a tx
```

Implementation note: write a single helper at `lib/db/withTenant.ts`. Every business-logic query must go through it. Direct Drizzle imports outside this helper are a code review red flag.

2.4 Audit logging (build day one)

`admin_audit_log` table tracks every cross-tenant read or write performed by the super admin app. Every admin query writes a row. Impersonation flows (super admin viewing as a tenant) display a banner in the UI and log start/end events.

2.5 Out of scope for v1

- Per-tenant database provisioning
- Cross-region data residency
- BYOK / tenant-managed encryption keys
- Complex tenant hierarchies (parent/child orgs)

---

3. Database schema

3.1 Conventions

- All IDs are `uuid` (`gen_random_uuid()` default).
- All tenant-scoped tables: `tenant_id uuid not null`, with an index. Composite indexes lead with `tenant_id`.
- Timestamps: `created_at` and `updated_at` are `timestamptz not null default now()`.
- Soft delete is **not** used in v1 (lean). Add `deleted_at` only when a feature requires it.
- Enums are Postgres enum types where stable; otherwise `text` with a check constraint.
- Channel-related columns are designed for SMS now and additive for email later — see table notes.

3.2 Tables

`tenants`

One row per customer business (1:1 with a Clerk Organization).

```sql
create table tenants (
  id                      uuid primary key default gen_random_uuid(),
  clerk_org_id            text unique not null,    -- maps to Clerk Org
  name                    text not null,           -- "Glow Aesthetics Las Vegas"
  subdomain               text unique,             -- nullable; set when tenant opt
  timezone                text not null default 'America/Los_Angeles',
  twilio_from_number      text,                    -- E.164, manually provisioned i
  twilio_account_sid_ref  text,                    -- if per-tenant subaccount used
  settings                jsonb not null default '{}'::jsonb,  -- spa_name, provide
  status                  text not null default 'active' check (status in ('active'
  created_at              timestamptz not null default now(),
  updated_at              timestamptz not null default now()
);

create unique index idx_tenants_clerk_org on tenants(clerk_org_id);
create index idx_tenants_subdomain on tenants(subdomain) where subdomain is not null
```

`settings` jsonb holds the PRD's Settings tab equivalents (`spa_name`, `provider_name`, `booking_link`, `reactivation_offer`, `offer_expiry_date`, `business_hours_start`, `business_hours_end`, `send_days`, `escalation_phone`, `escalation_email`). Keeping these in jsonb avoids schema churn while we discover what's actually needed.

`subdomain` is nullable and unused in v1. It's added now as free insurance so the first customer who wants a vanity URL doesn't trigger a schema migration. Behavior is documented in §3.5.

`users`

Application users within a tenant. Mirrors Clerk users — Clerk is source of truth for auth, this table holds app-level role and tenant scope.

```sql
```

```sql
create table users (
  id                  uuid primary key default gen_random_uuid(),
  clerk_user_id       text unique not null,
  tenant_id           uuid not null references tenants(id) on delete cascade,
  email               text not null,
  full_name           text,
  role                text not null default 'operator' check (role in ('owner','oper
  created_at          timestamptz not null default now(),
  updated_at          timestamptz not null default now()
);

create index idx_users_tenant on users(tenant_id);
```

Note: super admin users live in a separate `admin_users` table (see below). Customer users never have role = 'admin'.

`patients`

PRD Master Patient List columns + multi-tenant + future-proof channel flags.

```sql
```

```sql
create table patients (
  id                      uuid primary key default gen_random_uuid(),
  tenant_id               uuid not null references tenants(id) on delete cascade,
  external_patient_id     text,                    -- patient_id from CRM/CSV (PRD
  first_name              text,
  last_name               text,
  phone                   text,                    -- E.164, nullable (email-only p
  email                   text,                    -- nullable, used in v2
  last_visit_date         date,
  last_service            text,
  enrollment_date         date,
  sequence_track          text check (sequence_track in ('60_day','90_day','120_day
  status                  text not null default 'new'
                            check (status in ('new','enrolled','replied','converte
                                              'opted_out','no_response','sequence_
  last_contacted_at       timestamptz,
  last_message_number     int,
  channel_last_used       text check (channel_last_used in ('sms','email')),
  replied                 boolean not null default false,
  booking_link_clicked    boolean not null default false,
  opted_out               boolean not null default false,
  opted_out_at            timestamptz,
  converted               boolean not null default false,
  estimated_revenue_cents int,                     -- store money as integer cents
  notes                   text,
  source                  text not null default 'csv'
                            check (source in ('csv','boulevard','mangomint','manua
  created_at              timestamptz not null default now(),
  updated_at              timestamptz not null default now()
);

create index idx_patients_tenant on patients(tenant_id);
create unique index idx_patients_tenant_phone
  on patients(tenant_id, phone) where phone is not null;
create unique index idx_patients_tenant_external
  on patients(tenant_id, external_patient_id) where external_patient_id is not null;
create index idx_patients_tenant_status on patients(tenant_id, status);
```

Uniqueness keys: `(tenant_id, phone)` and `(tenant_id, external_patient_id)` — both partial (only when value is non-null). This lets us dedupe CSV imports safely while allowing patients without a phone (email-only in v2).

`conversations`

One conversation per patient. Channel-agnostic from day one — does **not** key on phone number.

sql

```sql
create table conversations (
  id                       uuid primary key default gen_random_uuid(),
  tenant_id                uuid not null references tenants(id) on delete cascade,
  patient_id               uuid not null references patients(id) on delete cascade,
  last_activity_at         timestamptz not null default now(),
  last_message_preview     text,
  last_message_direction   text check (last_message_direction in ('inbound','outboun
  unread_count             int not null default 0,
  created_at               timestamptz not null default now(),
  updated_at               timestamptz not null default now()
);

create unique index idx_conversations_tenant_patient on conversations(tenant_id, pat
create index idx_conversations_tenant_activity on conversations(tenant_id, last_acti
```

`messages`

Unified message log. SMS in v1, email additively in v2. Schema must not require migration to add email.

sql

```sql
create table messages (
  id                  uuid primary key default gen_random_uuid(),
  tenant_id           uuid not null references tenants(id) on delete cascade,
  conversation_id     uuid not null references conversations(id) on delete cascade,
  patient_id          uuid not null references patients(id) on delete cascade,

  channel             text not null check (channel in ('sms','email')),
  direction           text not null check (direction in ('inbound','outbound')),

  body_text           text not null,                  -- always populated
  body_html           text,                           -- v2 email
  subject             text,                           -- v2 email

  -- SMS-specific
  twilio_sid          text unique,                    -- MessageSid; idempotency ke
  twilio_from         text,
  twilio_to           text,
  segments            int,                            -- billing/segments from Twil
  price_cents         int,                            -- mirrored from Twilio

  -- email-specific (v2)
  email_message_id    text,
  email_in_reply_to   text,
  email_references    text,

  status              text not null default 'queued'
                          check (status in ('queued','sent','delivered','undelivered
                                            'failed','received','bounced','complaine
  status_updated_at   timestamptz,
  error_code          text,                           -- Twilio error code or email

  sent_by_user_id     uuid references users(id),      -- null if automated / inboun
  sent_at             timestamptz,                    -- when handed to provider
  read_at             timestamptz,                    -- when operator marked read

  created_at          timestamptz not null default now()
);

create index idx_messages_tenant_conv on messages(tenant_id, conversation_id, create
create index idx_messages_tenant_patient on messages(tenant_id, patient_id, created_
```

```sql
create index idx_messages_status_pending on messages(status)
  where status in ('queued','sent');  -- for nightly reconciliation
```

The status enum includes email statuses (`bounced`, `complained`, `received`) from day one. Adding email later is a new channel value + a new send function + a Resend webhook — zero data migration.

`csv_imports`

Audit + visible upload history for the CSV upload feature.

sql

```sql
create table csv_imports (
  id                  uuid primary key default gen_random_uuid(),
  tenant_id           uuid not null references tenants(id) on delete cascade,
  uploaded_by_user_id uuid not null references users(id),
  filename            text not null,
  total_rows          int not null,
  imported_count      int not null default 0,
  skipped_count       int not null default 0,
  errors              jsonb not null default '[]'::jsonb,  -- array of {row, reason}
  status              text not null default 'processing'
                          check (status in ('processing','complete','failed')),
  created_at          timestamptz not null default now(),
  completed_at        timestamptz
);

create index idx_csv_imports_tenant on csv_imports(tenant_id, created_at desc);
```

`admin_users`

Separate from `users`. Internal staff for the super admin app. Lives in the same DB but is accessed only by the `app_admin` Postgres role.

sql

```
create table admin_users (
  id              uuid primary key default gen_random_uuid(),
  clerk_user_id   text unique not null,            -- from the ADMIN Clerk instance
  email           text not null,
  full_name       text,
  role            text not null default 'admin' check (role in ('admin','support')),
  created_at      timestamptz not null default now()
);
```

`admin_audit_log`

Every cross-tenant access by the admin app writes here. Build day one.

sql

```
create table admin_audit_log (
  id                  uuid primary key default gen_random_uuid(),
  admin_user_id       uuid not null references admin_users(id),
  tenant_id_accessed  uuid references tenants(id),   -- null for tenant-list views
  action              text not null,                  -- 'view_tenant','impersonate
  resource            text,                           -- e.g. 'patients', 'messages
  resource_id         uuid,                           -- the specific record viewed
  metadata            jsonb not null default '{}'::jsonb,
  ip_address          inet,
  created_at          timestamptz not null default now()
);

create index idx_admin_audit_user_created on admin_audit_log(admin_user_id, created_
create index idx_admin_audit_tenant_created on admin_audit_log(tenant_id_accessed, c
```

3.3 Row-Level Security policies

Enable RLS on every tenant-scoped table. Customer app connects as `app_user`; admin app connects as `app_admin`.

sql

```
-- One-time setup per tenant-scoped table (run for tenants, users, patients,
-- conversations, messages, csv_imports):

alter table patients enable row level security;

-- Customer app: tenant-scoped
create policy patients_tenant_isolation on patients
  for all
  to app_user
  using (tenant_id = current_setting('app.current_tenant', true)::uuid)
  with check (tenant_id = current_setting('app.current_tenant', true)::uuid);

-- Admin app: read all, write rare and explicit
create policy patients_admin_all on patients
  for all
  to app_admin
  using (true)
  with check (true);

-- tenants table itself: customer app sees only its own row
create policy tenants_self on tenants
  for select
  to app_user
  using (id = current_setting('app.current_tenant', true)::uuid);
```

Apply the same pattern to `users`, `conversations`, `messages`, `csv_imports`. The `tenants` table's own RLS limits `app_user` to seeing only its own row.

3.4 The `withTenant` helper

typescript

```ts
// lib/db/withTenant.ts
import { db } from "./client";
import { sql } from "drizzle-orm";

export async function withTenant<T>(
  tenantId: string,
  fn: (tx: typeof db) => Promise<T>
): Promise<T> {
  return await db.transaction(async (tx) => {
    await tx.execute(sql`SELECT set_config('app.current_tenant', ${tenantId}, true)`
    return await fn(tx);
  });
}

// Usage in a Server Action / Route Handler:
const { orgId } = await auth();
const patients = await withTenant(orgId, (tx) =>
  tx.select().from(patientsTable).where(eq(patientsTable.status, 'enrolled'))
);
```

`set_config(..., true)` makes the setting transaction-local (`SET LOCAL` equivalent). RLS reads it, returns only matching rows, and the setting is discarded at commit. There is no way to accidentally leak a query without setting tenant context — the RLS policy rejects unset queries.

3.5 Subdomains (optional, off in v1)

Tenancy is enforced by Clerk + RLS, **not by the URL**. The `subdomain` column is reserved infrastructure for future vanity URLs — it does **not** participate in tenant isolation. The v1 build does not read or write this column.

Behavior model when subdomains are eventually wired up (v2 or whenever first customer requests it):

Tenant configuration	URL the tenant uses	How tenant is resolved
`subdomain = NULL`	`app.aura.app` (shared)	After sign-in: Clerk `orgId` → `tenants.clerk_org_id`
`subdomain = 'glowlv'`	`glowlv.aura.app`	Before sign-in: middleware looks up `tenants.subdomain`. Sign-in cross-checks that Clerk `orgId` matches.

Mixed deployments are supported by design: some tenants on the shared host, others on vanity subdomains, no forced migration when a tenant opts in or out. The v1 sign-in flow at `app.aura.app` continues working unchanged for every tenant indefinitely; vanity subdomains are pure additive UX.

v1 build action: leave the column nullable, do not populate it, do not read it. No middleware logic, no DNS work, no UI for selecting a subdomain. When the first customer asks, the work is ~1 day: a middleware change, wildcard DNS on the host, and a small admin UI to assign a subdomain to a tenant. Acceptance criteria for that follow-on work belong in a future spec, not this one.

---

4. Authentication

4.1 Two Clerk instances

Instance	Used by	Sign-in URL
Customer Clerk	Customer app (patient reactivation portal)	e.g. `app.aura.app/sign-in`
Admin Clerk	Super admin app (internal Anthropic staff)	e.g. `admin.aura.app/sign-in`

Separate Clerk instances were chosen for cleanest long-term isolation: zero risk of internal staff appearing as a customer org member, separate audit trail, separate billing visibility, separate sign-in branding. The added cost (~2x Clerk) is justified for the security and audit clarity.

4.2 Customer app auth

- Clerk Organizations enabled. Every signed-in user must belong to exactly one Organization.
- Clerk Organization `id` maps 1:1 to `tenants.clerk_org_id`. The first time a new org signs in, middleware lazily provisions a `tenants` row.
- Clerk middleware in `middleware.ts` protects every route except `/sign-in`, `/sign-up`, and webhook routes (e.g. `/api/twilio/webhook`, which are signature-verified instead).
- In any Server Component, Server Action, or Route Handler, `await auth()` returns `{ userId, orgId }`. `orgId` is the tenant id.
- Clerk webhooks (`/api/clerk/webhook`) sync `user.created`, `organization.created`, and `organizationMembership.created` events into the local `tenants` and `users` tables.

4.3 Admin app auth

- Separate Next.js app, separate repo (or a separate workspace package within a monorepo —

your call).

- Uses the Admin Clerk instance. Sign-in is restricted by Clerk allowlist to `@anthropic.com` or internal-domain emails.
- Connects to Neon as `app_admin` role.
- Read-only by default for tenant data. Write/impersonation flows are explicit, gated, and audited.

---

5. CSV upload → patients

5.1 Constraints (locked-in)

- v1 target: ⩽ 5 MB, ~10,000 rows. Synchronous parse in the API route (no background job needed at this size).
- Parser: `papaparse`, server-side. No client-side parsing.
- File itself is ephemeral — parse, insert, discard. No blob storage in v1.
- Dedupe key: `(tenant_id, phone)` primarily; `(tenant_id, external_patient_id)` as secondary.

5.2 Expected CSV columns

Accept the PRD's Master Patient List headers. Be lenient on case and underscore/space variations (`"First Name"`, `"first_name"`, `"FirstName"` all map). Required: at minimum one of (phone) or (email). All others optional.

5.3 Upload flow

```
User clicks "Upload CSV" in Patients screen
  |
  ▼
Browser: <input type="file" accept=".csv">  →  FormData POST
  |
  ▼
Route Handler:  POST /api/imports/csv
  ├─ auth() → tenantId, userId
  ├─ Read file (max 5 MB; reject larger with 413)
  ├─ INSERT csv_imports row (status='processing', total_rows=0)
  ├─ Parse with papaparse (header:true, skipEmptyLines:true)
  ├─ For each row:
  |     - Normalize header keys (lower, strip non-alphanum)
  |     - Normalize phone to E.164 (libphonenumber-js)
  |     - Validate: must have phone or email
  |     - If exists by (tenant_id, phone) or (tenant_id, external_patient_id):
  |         → skip, record reason 'duplicate' in errors[]
  |     - Else:
  |         → insert patients row with source='csv', status='new'
```

```
|         → imported_count++
├─ UPDATE csv_imports (status='complete', counts, errors[], completed_at)
└─ Return 200 { imported, skipped, errors[] }
```

5.4 Error handling rules

- Missing required columns: return 400 with a list of missing column names. Do not partially import.
- Per-row failures (bad phone format, duplicate, invalid email): skip the row, increment `skipped_count`, append `{ row_index, reason }` to `errors[]`. Continue processing the rest.
- Whole-file failures (corrupt file, encoding issue): mark `csv_imports.status = 'failed'`, return 422 with explanation.
- Idempotency: re-uploading the same file produces 0 imported, all rows skipped as duplicates. This is the correct behavior — don't add an idempotency token in v1.

5.5 UI behavior

Patients page (per `index.html` reference):

- Top-right primary button: **"Upload CSV"**. Opens a modal with drag-drop area and template-download link.
- On submit: progress spinner. On 200: toast *"Imported X patients, skipped Y"*. Table refreshes.
- If `errors[]` non-empty: expandable panel in the result toast showing first 10 errors and "download full error report" (CSV of row+reason).
- Patients table columns reflect the schema: Name, Phone, Email, Last visit, Status, Source. Filter by status. Sort by `last_visit_date`.

---

6. Twilio inbox (SMS send + receive)

6.1 Configuration

- One Twilio number per tenant. Provisioned manually in v1 and stored in `tenants.twilio_from_number`.
- Twilio webhook URL (inbound + status callback) configured per number in the Twilio console to point at `/api/twilio/webhook` and `/api/twilio/status` respectively. Use a single Twilio main account; subaccounts deferred.
- Required env vars: `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `PUBLIC_URL` (used for signature validation — do **not** reconstruct from request).

6.2 Outbound send flow

```
Operator clicks Send in inbox compose box
  |
  ▼
Server Action: sendMessage({ conversationId, body })
  ├─ auth() → tenantId, userId
  ├─ Load conversation + patient (tenant-scoped via withTenant)
  ├─ HARD CHECK: patient.opted_out === false  (compliance, no exceptions)
  ├─ INSERT messages row (status='queued', channel='sms', direction='outbound',
  |                       body_text, sent_by_user_id=userId)
  ├─ Twilio API: client.messages.create({
  |       to: patient.phone,
  |       from: tenant.twilio_from_number,
  |       body: body,
  |       statusCallback: `${PUBLIC_URL}/api/twilio/status`
  |     })
  ├─ UPDATE messages SET twilio_sid=<sid>, status='sent', sent_at=now()
  ├─ UPDATE conversations SET last_activity_at=now(), last_message_preview,
last_message_direction='outbound'
  └─ Emit SSE event to inbox subscribers for this tenant
```

The `opted_out` check is performed at send time, not at enrollment time. This is a compliance requirement — sending after STOP is a critical violation.

6.3 Inbound webhook

```
POST /api/twilio/webhook
  ├─ Validate X-Twilio-Signature using twilio.validateRequest
  |     with PUBLIC_URL env var (NOT reconstructed from request headers)
  ├─ Reject 403 if invalid
  ├─ Parse form-encoded body: From, To, Body, MessageSid, NumMedia, etc.
  ├─ Resolve tenant by matching To (the Twilio number) →
tenants.twilio_from_number
  ├─ withTenant(tenant.id, async (tx) => {
  |     - Idempotency: SELECT from messages WHERE twilio_sid = MessageSid
  |         → if exists, return 200 immediately (Twilio retries)
  |     - Find patient by (tenant_id, phone = From). If missing, create a new
  |         patients row with source='manual', status='new'.
  |     - Upsert conversation for that patient.
  |     - Insert messages row (channel='sms', direction='inbound',
  |         body_text=Body, twilio_sid=MessageSid, status='received').
```

```
  |     - Update conversations (last_activity_at, last_message_preview,
  |         unread_count++, last_message_direction='inbound').
  |     - Reply handler (see 6.5).
  |   })
  ├─ Emit SSE event to inbox subscribers
  └─ Respond 200 with empty TwiML  (<Response/>)
```

Webhook must return 200 within 500ms. Any heavy work (notifications, LLM intent detection in v2) is offloaded to Inngest from inside the handler.

6.4 Status callback

```
POST /api/twilio/status
  ├─ Validate X-Twilio-Signature
  ├─ Parse MessageSid + MessageStatus + ErrorCode
  ├─ UPDATE messages SET status=<mapped>, status_updated_at=now(), error_code=<>
  |     WHERE twilio_sid = MessageSid
  └─ 200 OK

Twilio status → local status mapping:
  queued       → queued
  sending/sent → sent
  delivered    → delivered
  undelivered  → undelivered
  failed       → failed
```

6.5 Reply handler (v1 keyword matching)

Implemented inside the inbound webhook handler, after the message is recorded. Returns a structured intent so v2 LLM swap is a function-level change.

typescript

```
function detectIntent(body: string): { intent: string; confidence: 'high'|'low' } {
  const normalized = body.trim().toLowerCase();

  // STOP variants — high confidence
  if (/^(stop|stopall|unsubscribe|cancel|end|quit)\b/.test(normalized)) {
    return { intent: 'opt_out', confidence: 'high' };
  }

  // Booking intent keywords
  const bookingKw = ['book','appointment','schedule','available','when','price','cos
  if (bookingKw.some(kw => normalized.includes(kw))) {
    return { intent: 'booking_interest', confidence: 'low' };  // low until LLM in v
  }

  return { intent: 'other', confidence: 'low' };
}

// After inserting the inbound message:
const { intent } = detectIntent(message.body_text);
switch (intent) {
  case 'opt_out':
    // UPDATE patients SET opted_out=true, opted_out_at=now(), status='opted_out'
    //   WHERE id = patient.id
    // send Twilio confirmation: "You have been unsubscribed. Reply START to resubsc
    break;
  case 'booking_interest':
    // UPDATE patients SET replied=true, status='replied'
    // emit SSE 'hot_lead' for inbox UI
    // (escalation alerts deferred to later — log only in v1)
    break;
  case 'other':
    // UPDATE patients SET replied=true, status='replied'
    break;
}
```

6.6 Real-time inbox via SSE

```
GET /api/inbox/stream  (runtime: 'edge' if possible, else 'nodejs')
  ├─ auth() → tenantId
  ├─ Set headers: Content-Type: text/event-stream
  |               Cache-Control: no-cache, no-transform
  |               Connection: keep-alive
```

```
  ├─ Subscribe to in-memory event bus keyed by tenantId
  ├─ Stream events as: data:
{"type":"message.received","conversationId":"..."}\n\n
  ├─ Heartbeat ping every 25s to keep connection alive
  └─ Client reconnects every ~10 min to avoid serverless function timeout

Event bus (v1): simple in-process EventEmitter per Node instance.
  → Works for single-instance deploys (Vercel serverless on cold start = new
instance,
    but SSE connections naturally re-establish).
  → If we ever need cross-instance fanout, swap for Redis pub/sub or Postgres
LISTEN/NOTIFY.
    That swap is one file (lib/realtime/bus.ts).
```

Client-side: use the native `EventSource` API. On the inbox page, subscribe on mount, update React state on each event, unsubscribe on unmount. Reconnect on error after exponential backoff (1s, 2s, 4s, max 30s).

6.7 Inbox UI (per `index.html` reference)

- Three-pane layout: conversation list (left, 380px) / thread (right, fills). Active row highlighted with `--accent-soft`.
- Unread rows show a dot indicator. Click to open thread; clears `unread_count`.
- Thread renders message bubbles. Bubble component accepts a `channel` prop and renders SMS styling in v1; email styling added in v2 without touching the parent.
- Compose box at bottom: single textarea, character counter, Send button. Show segment count if body > 160 chars.
- Top of thread: patient name, phone, status badge, link to patient profile.
- Empty state when no conversation selected: *"Select a conversation"*.

6.8 Reconciliation (nightly via Inngest)

One Inngest scheduled function, runs nightly at 03:00 UTC. Queries Twilio Messages API for the last 24–48h, compares to local `messages.status`, fixes any drift from missed webhooks. Single function, ~50 LOC. Defer until inbox is functioning.

---

7. Other portal screens (static UI per `index.html`)

Build the visual shell of these screens matching the `index.html` reference. Wiring to real data is optional in this build pass — render with placeholder/seeded data is acceptable so we can demo the full portal feel.

Screen	Source	v1 wiring
Dashboard / Home	`index.html` KPIs + funnel + activity log	Static (placeholder numbers)
Patients	`index.html` patients table	**WIRED** — reads `patients` table
Inbox	`index.html` inbox-grid	**WIRED** — full Twilio integration
Sequences	`index.html` sequence builder/timeline	Static (display only)
Settings	`index.html` field-row settings layout	Static + load/save `tenants.settings` jsonb
Connections	`index.html` conn-card grid	Static + Twilio status badge from env
Activity log	`index.html` log-line list	**WIRED** — reads recent messages + csv_imports

Use the exact CSS variables (`--accent`, `--panel`, `--border`, etc.) and class names from `index.html` so the design system is consistent. The reference uses Geist as the primary font — keep that, plus Geist Mono for tabular data and code-like values.

---

8. Project structure

```
aura-app/                          # customer app
├─ app/
│  ├─ (auth)/sign-in/[[...rest]]/page.tsx
│  ├─ (auth)/sign-up/[[...rest]]/page.tsx
│  ├─ (portal)/
│  │  ├─ layout.tsx                # sidebar + topbar shell (per index.html)
│  │  ├─ page.tsx                  # dashboard
│  │  ├─ patients/page.tsx
│  │  ├─ patients/[id]/page.tsx
│  │  ├─ inbox/page.tsx            # three-pane SSE-powered inbox
│  │  ├─ sequences/page.tsx        # static in v1
│  │  ├─ settings/page.tsx
│  │  └─ connections/page.tsx
│  └─ api/
│     ├─ imports/csv/route.ts      # POST: CSV upload handler
```

```
|     ├─ inbox/stream/route.ts      # GET: SSE endpoint
|     ├─ messages/send/route.ts     # (optional, Server Action preferred)
|     ├─ twilio/webhook/route.ts    # POST: inbound SMS
|     ├─ twilio/status/route.ts     # POST: status callback
|     └─ clerk/webhook/route.ts     # POST: user/org sync
├─ components/
|  ├─ portal/Sidebar.tsx
|  ├─ portal/Topbar.tsx
|  ├─ inbox/ConversationList.tsx
|  ├─ inbox/Thread.tsx
|  ├─ inbox/MessageBubble.tsx       # accepts channel prop, forward-compatible
|  ├─ inbox/Composer.tsx
|  └─ patients/UploadCsvModal.tsx
├─ lib/
|  ├─ db/
|  |  ├─ client.ts                  # Drizzle client (app_user role)
|  |  ├─ schema.ts                  # All tables defined here
|  |  └─ withTenant.ts              # The tenant-scoped tx helper
|  ├─ services/
|  |  ├─ patients.ts                # importCsv, listPatients, getPatient
|  |  ├─ conversations.ts           # listConversations, getThread
|  |  ├─ messages.ts                # sendMessage, recordInbound
|  |  └─ tenants.ts                 # getTenantBySid, getTenantByOrgId
|  ├─ twilio/
|  |  ├─ client.ts
|  |  ├─ verify.ts                  # validateRequest wrapper
|  |  └─ intent.ts                  # detectIntent (v1 keyword, v2 LLM)
|  ├─ realtime/
|  |  └─ bus.ts                     # in-process EventEmitter (swap for Redis
later)
|  └─ csv/
|     └─ parse.ts                   # papaparse wrapper + normalizers
├─ middleware.ts                    # Clerk middleware
├─ drizzle/                         # migrations
├─ inngest/
|  └─ functions/
|     └─ reconcile-twilio.ts        # nightly status reconciliation
├─ drizzle.config.ts
├─ next.config.ts                   # output: 'standalone'
└─ .env.example

aura-admin/                         # super admin app (separate repo or workspace)
├─ app/
|  ├─ (auth)/sign-in/[[...rest]]/page.tsx
```

```
|  ├─ tenants/page.tsx              # list all tenants
|  ├─ tenants/[id]/page.tsx         # tenant detail (read-only)
|  └─ audit/page.tsx                # admin_audit_log viewer
├─ lib/
|  ├─ db/client.ts                  # Drizzle client (app_admin role)
|  └─ services/
|     ├─ tenants.ts
|     └─ audit.ts
└─ middleware.ts                   # Admin Clerk middleware
```

---

9. Environment variables

9.1 Customer app

bash

```
# --- Database ---
DATABASE_URL=postgres://app_user:...@...neon.tech/aura?sslmode=require

# --- Clerk (customer instance) ---
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...
CLERK_SECRET_KEY=sk_test_...
CLERK_WEBHOOK_SECRET=whsec_...

# --- Twilio ---
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
PUBLIC_URL=https://app.aura.app   # used for signature validation; set in dev to ngr

# --- Inngest ---
INNGEST_EVENT_KEY=...
INNGEST_SIGNING_KEY=signkey_...

# --- App ---
NODE_ENV=development
```

9.2 Admin app

bash

```
DATABASE_URL=postgres://app_admin:...@...neon.tech/aura?sslmode=require

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...    # ADMIN Clerk instance
CLERK_SECRET_KEY=sk_test_...                     # ADMIN Clerk instance

PUBLIC_URL=https://admin.aura.app
```

---

10. Local development

- Node 20+, `pnpm` (or npm).
- Neon: create a dev branch. Run migrations: `pnpm drizzle-kit push`.
- RLS setup: after migrations, run a seed SQL file that creates the `app_user` and `app_admin` roles and applies the RLS policies. Commit this as `drizzle/seeds/01-roles.sql`.
- ngrok or cloudflared tunnel for Twilio webhooks. Set `PUBLIC_URL` to the tunnel URL. Update the Twilio number webhook in the console (or use `twilio phone-numbers:update`).
- Signature validation MUST be on in dev. Never bypass.
- Seed data: a `drizzle/seeds/02-dev-tenant.sql` file creates one dev tenant + one user mapped to the Clerk dev user id. Document the steps in README.

---

11. Acceptance criteria (v1)

This build is correct when all of the following pass on a clean check-out:

Database & multi-tenancy

- [ ] Migrations apply cleanly to an empty Neon database.
- [ ] `app_user` and `app_admin` Postgres roles exist with the documented RLS policies.
- [ ] With tenant context unset, `app_user` sees **zero** rows from any tenant-scoped table.
- [ ] With tenant A's context set, `app_user` sees only tenant A's rows. Tenant B's rows are invisible. Verified with a test query in two separate transactions.
- [ ] `app_admin` sees all rows across all tenants.

Auth

- [ ] Signing in to the customer app as a new user creates a Clerk Organization, a `tenants` row, and a `users` row via the Clerk webhook.

- [ ] Signing in to the admin app from a non-allowed email is rejected by Clerk.
- [ ] `middleware.ts` protects all portal routes; webhook routes are accessible without Clerk session (verified by signature instead).

CSV upload

- [ ] Uploading a 20-row valid CSV inserts 20 patients with `status='new'`, `source='csv'`.
- [ ] Re-uploading the same file produces 0 imported, 20 skipped (duplicates by phone).
- [ ] Uploading a file with one bad row (invalid phone) inserts 19, skips 1, `errors[]` contains the row index and reason.
- [ ] A `csv_imports` row is recorded with correct counts and `status='complete'`.
- [ ] Files > 5 MB are rejected with 413.

Twilio inbox

- [ ] Sending an SMS from the compose box creates a `messages` row with `channel='sms'`, `direction='outbound'`, `status='queued'`, then updates to `status='sent'` with a `twilio_sid`.
- [ ] Status callback updates `messages.status` to `'delivered'` when Twilio reports delivery.
- [ ] An inbound SMS to the tenant's Twilio number creates a `messages` row with `direction='inbound'`, upserts the conversation, and increments `unread_count`.
- [ ] Sending the same inbound webhook twice (Twilio retry) results in only one `messages` row (idempotent on `twilio_sid`).
- [ ] Replying STOP sets the patient's `opted_out=true` and `status='opted_out'`, and sends a Twilio confirmation.
- [ ] Attempting to send to an opted-out patient is rejected with a clear error before any Twilio API call is made.
- [ ] The inbox UI updates in real-time when a new inbound message arrives (SSE).
- [ ] `X-Twilio-Signature` validation is enabled in dev and prod; requests with bad signatures return 403.

Other screens

- [ ] Sidebar + topbar + page shell match the `index.html` reference visually (colors, spacing, fonts).
- [ ] Dashboard, Sequences, Settings, Connections render as static screens with the reference design.
- [ ] Patients table reads from the `patients` table and supports basic sorting/filtering.

---

12. v2 forward-compatibility (do not build now)

The v1 build is correct only if these v2 changes require **additive** work, not migrations or refactors:

- **Email channel via Resend:** a new `channel='email'` value, a `sendEmail` function, a `/api/resend/webhook` route. Schema is already ready (`body_html`, `subject`, `email_message_id`, `in_reply_to`, `references` columns exist).
- **LLM intent detection:** replace `detectIntent()` body with an LLM call returning `{ intent, entities, confidence }`. Caller signature unchanged.
- **Booking adapter:** add `lib/booking/adapter.ts` interface with `getBookingLink(patient, context)` in v1; `getAvailability(date)` and `holdSlot(slotId)` added later. Adapter impls in `lib/booking/boulevard.ts`, `lib/booking/calendly.ts`, etc.
- **Sequence execution engine:** an Inngest scheduled function that reads patients with `status='enrolled'`, computes `days_since_enrollment`, sends the message for that day. Reuses `sendMessage`. No schema changes.
- **Real-time scale:** when in-process `EventEmitter` is insufficient (multi-instance deploys), swap `lib/realtime/bus.ts` for Postgres LISTEN/NOTIFY or Redis pub/sub. Single file change.
- **Reporting & digests:** a nightly Inngest job + a transactional email send. No schema changes.
- **Vanity subdomains** (e.g. `glowlv.aura.app`): add subdomain-extraction logic to middleware, configure wildcard DNS on the host, and add an admin UI to assign a subdomain to a tenant. The `tenants.subdomain` column already exists from v1. Tenants without a subdomain continue working at `app.aura.app` unchanged — mixed deployments are supported by design.

---

13. Key gotchas — don't skip

- **Twilio signature validation:** pass `PUBLIC_URL` env var explicitly to `validateRequest`. Do **not** reconstruct the URL from request headers — proxies (Vercel, Cloudflare, ngrok) rewrite Host and the signature breaks.
- **Twilio MessageSid is the idempotency key for inbound webhooks.** Always `SELECT` before `INSERT`. Twilio retries on non-2xx and on timeouts.
- **`opted_out` is checked before EVERY send,** not at enrollment. Wrapping this check in the `sendMessage` service is required.
- **RLS only works when the connection is the right role.** Pooled connections must use `app_user`. The admin app uses its own pool with `app_admin`. Do not share a Drizzle client

between them.

- `set_config('app.current_tenant', ..., true)` — the third argument `true` makes it transaction-local. Without `true`, the setting persists in the connection and leaks across requests in a connection pool.
- **Webhook routes must NOT be inside the Clerk middleware matcher.** Add them to the public matcher list explicitly.
- **E.164 normalization on import:** use `libphonenumber-js` with a default country (read from `tenant.settings` or default `'US'`). Reject rows where normalization fails.
- **Twilio's standard 10DLC throughput is 1 SMS/sec per number.** Bulk-send features (when added in v2) must respect this — don't blast hundreds of messages synchronously.

---

14. Open questions for follow-up sessions

- Per-tenant Twilio subaccounts vs single account: defer until first paying customer.
- A2P 10DLC registration workflow — who handles the brand/campaign filings: us or the tenant?
- Compliance / data retention policy — HIPAA is med spa adjacent but not strictly required. Decide before first real patient data is loaded.
- Onboarding flow: how does a new tenant get its Twilio number assigned? Manual for first 5 tenants; automation in v2.
- Pricing model — affects whether tenants need usage metering tables in v2.

— End of document —